Nagar Yuwak Shikshan Sanstha's
Yeshwantrao Chavan College of Engineering
(An Autonomous Institution affiliated to Rashtrasant Tukadoji Maharaj Nagpur University)
Hingna Road, Wanadongri, Nagpur - 441 110
NAAC A++
Ph.: 07104-237919, 234623, 329249, 329250 Fax: 07104-232376, Website: www.ycce.edu

Department of Computer Technology

**Vision of the Department**

*To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.*

**Mission of the Department**

*To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.*

**Session 2025-2026**

| | |
|---|---|
| **Vision:** Dream of where you want. | **Mission:** Means to achieve Vision |

**Program Educational Objectives of the program (PEO):** (broad statements that describe the professional and career accomplishments)

| | | | |
|---|---|---|---|
| PEO1 | **Preparation** | **P: Preparation** | **Pep-CL abbreviation pronounce as Pep-si-lL easy to recall** |
| PEO2 | **Core Competence** | **E: Environment (Learning Environment)** | |
| PEO3 | **Breadth** | **P: Professionalism** | |
| PEO4 | **Professionalism** | **C: Core Competence** | |
| PEO5 | **Learning Environment** | **L: Breadth (Learning in diverse areas)** | |

**Program Outcomes (PO):** (statements that describe what a student should be able to do and know by the end of a program)

**Keywords of POs:**
Engineering knowledge, Problem analysis, Design/development of solutions, Conduct Investigations of Complex Problems, Engineering Tool Usage, The Engineer and The World, Ethics, Individual and Collaborative Team work, Communication, Project Management and Finance, Life-Long Learning

**PSO Keywords:** Cutting edge technologies, Research

"I am an engineer, and I know how to apply engineering knowledge to investigate, analyse and design solutions to complex problems using tools for entire world following all ethics in a collaborative way with proper management skills throughout my life." *to contribute to the development of cutting-edge technologies and Research*.

**Integrity:** I will adhere to the Laboratory Code of Conduct and ethics in its entirety.

**Name and Signature of Student and Date**
(Signature and Date in Handwritten)

| **Session** | **2025-26 (Even)** | **Course Name** | **PE-III: Lab -Reinforcement Learning** |
|---|---|---|---|
| **Semester** | **6th** | **Course Code** | **23ADS1644** |
| **Roll No** | **13** | **Name of Student** | **Prerna Utane** |

| | |
|---|---|
| Practical Number | **05** |
| Course Outcome | CO1: Apply fundamental reinforcement learning concepts to model and solve decision-making problems.<br>CO2: Implement basic reinforcement learning algorithms to train agents in a given<br>environment.<br>CO3: Analyze agent behavior |
| Aim | To implement a multi-armed bandit problem to study the exploration-exploitation trade-off. |
| Problem Definition | To implement a **Multi-Armed Bandit** problem using the **Epsilon-Greedy algorithm** in order to study the **exploration vs. exploitation trade-off**. The goal is to maximize cumulative rewards by selecting optimal actions over multiple rounds while balancing between trying new actions (exploration) and using known best actions (exploitation). |
| Theory (100 words) | The Multi-Armed Bandit problem is a fundamental concept in reinforcement learning where an agent must choose between multiple actions (arms) with uncertain rewards. Each arm provides a reward based on an unknown probability distribution. The challenge lies in balancing exploration (trying different arms to gather information) and exploitation (choosing the best-known arm to maximize reward). The Epsilon-Greedy strategy addresses this trade-off by selecting a random action with probability ε (exploration) and the best-known action with probability 1−ε (exploitation). Over time, the agent updates estimated rewards (Q-values) based on observed outcomes. This approach helps in converging toward the optimal action while still allowing occasional exploration to avoid suboptimal decisions. |

| | |
|---|---|
| Procedure and Execution<br><br>(100 Words) | Algorithm:<br>1. Initialize:<br>• Number of arms k<br>• Q-values for each arm = 0<br>• Action counts = 0<br>• Exploration rate ε<br>2. Load true mean rewards for each arm.<br>3. For each round (1 to N):<br>• Generate a random number.<br>• If random < ε:<br>  o Select a random arm (exploration)<br>• Else:<br>  o Select arm with highest Q-value (exploitation)<br>• Pull selected arm and receive reward.<br>• Update action count for that arm.<br>• Update Q-value using:<br>Q(a)=Q(a)+1(R−Q(a))/N<br>4. Track:<br>• Rewards obtained<br>• Optimal action selections<br>5. Compute:<br>• Cumulative average reward<br>• Percentage of optimal actions<br>6. Plot results. |
| | Code:<br><br>import numpy as np<br>import matplotlib.pyplot as plt<br>import pandas as pd |

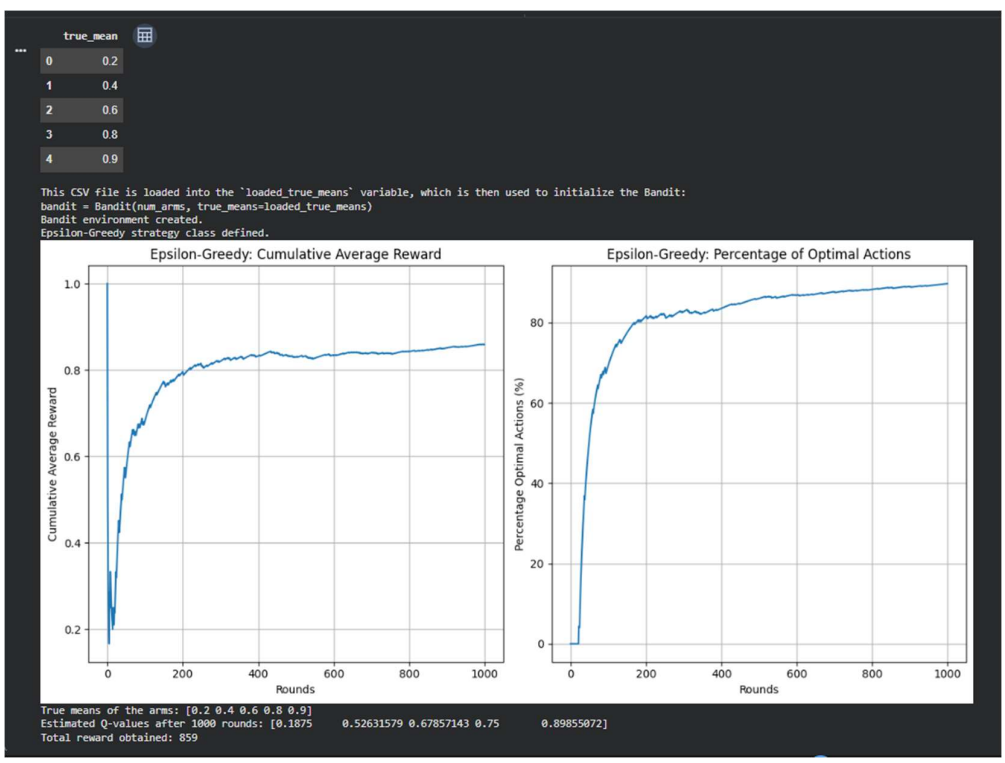
```
display(pd.read_csv('bandit_true_means.csv'))

print("\nThis CSV file is loaded into the `loaded_true_means` variable, which is
then used to initialize the Bandit:")
print("bandit = Bandit(num_arms, true_means=loaded_true_means)")

class Bandit:
    def __init__(self, num_arms, true_means=None):
        self.num_arms = num_arms
        if true_means is not None:
            if len(true_means) != num_arms:
                raise ValueError("Length of true_means must match num_arms")
            self.true_means = np.array(true_means)
        else:
            self.true_means = np.random.rand(num_arms)

    def pull(self, arm_index):
        if np.random.rand() < self.true_means[arm_index]:
            return 1
        else:
            return 0

    def get_true_means(self):
        return self.true_means

print("Bandit environment created.")
class EpsilonGreedy:
    def __init__(self, num_arms, epsilon):
        self.num_arms = num_arms
        self.epsilon = epsilon
        self.q_values = np.zeros(num_arms)
        self.action_counts = np.zeros(num_arms)

    def choose_action(self):
        if np.random.rand() < self.epsilon:
            return np.random.randint(self.num_arms)
        else:
            return np.argmax(self.q_values)

    def update(self, arm_index, reward):
        self.action_counts[arm_index] += 1
```

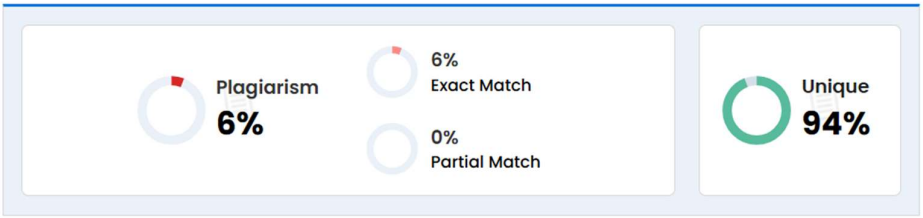
```
        alpha = 1 / self.action_counts[arm_index]
        self.q_values[arm_index] += alpha * (reward - self.q_values[arm_index])

print("Epsilon-Greedy strategy class defined.")
num_arms = 5
num_rounds = 1000
epsilon = 0.1

bandit = Bandit(num_arms, true_means=loaded_true_means)
agent = EpsilonGreedy(num_arms, epsilon)

rewards = []
optimal_actions = []

optimal_arm_index = np.argmax(bandit.get_true_means())

for i in range(num_rounds):
    action = agent.choose_action()
    reward = bandit.pull(action)
    agent.update(action, reward)

    rewards.append(reward)
    optimal_actions.append(1 if action == optimal_arm_index else 0)

cumulative_average_reward = np.cumsum(rewards) / (np.arange(num_rounds) +
1)

percentage_optimal_actions = np.cumsum(optimal_actions) /
(np.arange(num_rounds) + 1)

plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.plot(cumulative_average_reward)
plt.xlabel('Rounds')
plt.ylabel('Cumulative Average Reward')
plt.title('Epsilon-Greedy: Cumulative Average Reward')
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(percentage_optimal_actions * 100)
```

```
plt.xlabel('Rounds')
plt.ylabel('Percentage Optimal Actions (%)')
plt.title('Epsilon-Greedy: Percentage of Optimal Actions')
plt.grid(True)

plt.tight_layout()
plt.show()

print(f"True means of the arms: {bandit.get_true_means()}")
print(f"Estimated Q-values after {num_rounds} rounds: {agent.q_values}")
print(f"Total reward obtained: {np.sum(rewards)}")
```

Output:

1. **Epsilon Graph**



| | |
|---|---|
| Output Analysis | • **Cumulative Average Reward Graph**: Initially fluctuates due to exploration but gradually stabilizes as the agent learns the best arm.<br>• **Percentage of Optimal Actions Graph**: Starts low and increases over time, showing that the agent increasingly selects the optimal arm.<br>• **Q-values vs True Means**: Estimated Q-values converge close to the true means of each arm.<br>• **Effect of ε (0.1)**: Ensures 10% exploration, preventing the agent from getting stuck in suboptimal choices.<br>• **Total Reward**: Increases steadily, indicating successful learning. |
| Link of student Github profile where lab assignment has been uploaded | https://github.com/23071121-pixel/RL_Practicals/edit/main/Practical-5_13 |
| Conclusion | The implementation successfully demonstrates the exploration-exploitation trade-off using the Epsilon-Greedy strategy. The agent improves its performance over time by learning the optimal arm through repeated interactions. While exploration may reduce short-term rewards, it ensures better long-term performance by discovering higher-reward actions. The results confirm that a balanced ε value leads to effective learning and convergence toward optimal behavior in stochastic environments. |
| Plag Report (Similarity index < 12%) |  |
| Date | 28/03/2026 |